

Architectural Determinants of Behaviour in AI Coding Agents: A Concise Theoretical and Empirical Synthesis

Anand Venkat

Abstract—AI coding assistants built atop similar large language models frequently show markedly different capabilities on real, multi-file engineering work. This paper advances a compact, practically useful theory: four measurable *architectural determinants*—*contextual depth*, *execution scope*, *integration richness*, and *control topology*—predict an assistant’s behavioural profile. We summarise a focused 16-week empirical evaluation (five commercial assistants; six enterprise-style tasks; 149k LOC) validating the theory, derive a three-class behavioural taxonomy, and present two human–AI interaction paradigms. The equations and measurement heuristics included are intended to make the framework directly usable for engineers evaluating or designing coding assistants. A compact regression model using these four determinants explains 87 percent of observed autonomy variance across 30 tool-task combinations.”

EXECUTIVE SUMMARY

This paper explains why superficially similar AI coding assistants behave differently when asked to perform complex engineering tasks. The central insight is that architectural design choices (how much context a tool can use, what it is allowed to execute, how well it integrates with other systems, and how its internal reasoning is organised) determine practical capability more than base-model size alone. We provide a small set of equations and quick measurement heuristics that engineering teams can use to profile candidate tools, match tools to task types, and set appropriate governance. Our 16-week empirical evaluation shows these architectural features reliably predict autonomy and deployment success; practical guidance for both purchasers and designers is included.

Index Terms—AI-assisted software engineering, agent architectures, empirical evaluation, human–AI collaboration, behavioural taxonomy.

I. INTRODUCTION

Large language models (LLMs) have been integrated into developer tooling at scale. Yet, superficially similar assistants often behave very differently: some autonomously orchestrate multi-service deployments while others remain confined to inline completions. We propose that these differences arise primarily from architectural design — how context is managed, which actions are permitted, how integrations are engineered, and how internal control is organised — rather than from model scale alone.

Independent technology assessment. Artifacts and protocols available on request.

This abridged paper delivers: (1) a compact theoretical framework with simple equations that engineers can compute or approximate; (2) an empirical validation summary from a 16-week study; and (3) practical, rapid heuristics for procurement and design.

II. ARCHITECTURAL DETERMINANTS: A COMPACT FORMALISATION

We represent each tool T with an architectural profile vector

$$\mathbf{A}_T = [C_T, E_T, I_T, P_T],$$

where each component captures a distinct design lever that jointly shapes observable behaviour.

A. Contextual depth C_T

Contextual depth measures the assistant’s effective ability to reason across repo-scale context and to retain relevant information across interactions. It combines three elements: the static token window w_T , memory reliability m_T , and cross-artifact linking l_T . We use the form:

$$C_T = \log(1 + w_T) (0.5 + 0.3 m_T + 0.2 l_T). \quad (1)$$

Intuitive explanation: contextual depth is like a flashlight in a large attic. The beam width is the token window w_T , memory m_T is your notepad, and linking l_T connects clues across boxes. A larger beam helps, but real progress comes from reliable memory and connection-making.

B. Execution scope E_T

Execution scope is an ordinal variable describing the breadth of environment-facing actions:

$$E_T \in \{0, 1, 2, 3, 4\},$$

with 0 = text-only suggestions, 1 = file read/write, 2 = process execution, 3 = version-control operations, and 4 = container orchestration.

Intuitive explanation: execution scope is like granting a junior engineer permissions—from ‘suggest only’ (0) to ‘deploy to staging’ (4). The higher the level, the greater the autonomy—and the need for governance.

C. Integration richness I_T

Integration richness quantifies how well the assistant connects to external systems:

$$I_T = \sum_{k=1}^{N_T} \rho_{T,k}, \quad (2)$$

where $\rho_{T,k}$ is the empirical reliability of integration k .

Intuitive explanation: each successful integration is a bridge to another system. A few strong bridges are better than many that collapse under load. Summing their reliability gives total bridge capacity.

D. Control topology P_T

Control topology captures reasoning organisation:

- **Type A:** monolithic, single-pass
- **Type B:** iterative refinement
- **Type C:** multi-agent orchestration

Intuitive explanation: topology is the tool’s work style—solo coder (A), iterative reviewer (B), or small team (C).

III. COMPARING TOOLS: ARCHITECTURAL DISTANCE

$$d(T_i, T_j) = \left\| \text{norm}([C, E, I, p])_{T_i} - \text{norm}([C, E, I, p])_{T_j} \right\|_2. \quad (3)$$

Intuitive explanation: treat each tool as a point on a capability map; smaller distance means similar behaviour.

IV. PREDICTING AUTONOMY

$$A_u = \alpha_0 + \alpha_1 C_T + \alpha_2 E_T + \alpha_3 I_T + \alpha_4 P_T + \varepsilon. \quad (4)$$

Intuitive explanation: autonomy is like a scorecard—each determinant adds points toward self-sufficiency.

V. BEHAVIOURAL TAXONOMY AND ALIGNMENT

Three archetypes emerge: Agentic Execution Engine, Workspace-Bound Collaborator, and Reactive Completion Engine (Table I). Matching interaction paradigm to archetype optimises performance; mismatches cause coordination drag.

TABLE I
ARCHETYPE SUMMARY AND QUICK RULE-OF-THUMB

Archetype	Rule-of-thumb
Agentic Execution Engine	High C_T , $E_T \geq 3$, high I_T , Type C. Use for deployments and large transformations.
Workspace-Bound Collaborator	Moderate E_T , strong C_T , I_T , Type B. Best for documentation and refactors.
Reactive Completion Engine	Low C_T , E_T , I_T , Type A. Best for local edits.

VI. EMPIRICAL STUDY (CONCISE RECAP)

A 16-week evaluation across six enterprise tasks and five tools (Claude Code, Windsurf, GitHub Copilot, GitLab Duo, Junie, Amazon Q) showed determinant-based regression ($R^2 \approx 0.87$) predicted autonomy well. Agentic tools completed complex tasks with 0–2 interventions; reactive tools needed 7.

VII. REPRESENTATIVE VIGNETTES

Agentic success: A Type C assistant automated legacy migration and deployment with minimal oversight. **Reactive failure:** A Type A assistant produced partial edits but missed cross-file links, requiring manual rollback.

VIII. PRACTICAL RECOMMENDATIONS

For engineering teams: estimate A_T , match archetype to task, enforce sandboxed execution for high-autonomy tools. For designers: expand context and memory, raise execution safely, build robust connectors, and evolve toward Type C topologies.

IX. HEURISTIC ASSESSMENT GUIDE FOR PRACTITIONERS AND DESIGNERS

Detailed benchmarking can take weeks. A rapid heuristic pass—just five to ten minutes of observation—often predicts autonomy within a narrow band.

A. Quick observational cues (first 5 prompts)

- 1) **Prompt memory test:** Ask the tool to refactor two related files, then request a cross-reference explanation. If it connects them unaided, C_T is high. *Example:* Claude Code and Windsurf recall file relationships; Copilot and Amazon Q forget context quickly.
- 2) **Action test:** Ask it to “run unit tests” or “commit this change.” If it replies with executable steps or performs them, $E_T \geq 2$. *Example:* Windsurf’s built-in shell passes Level 2; GitHub Copilot remains Level 0–1.
- 3) **Integration probe:** Request external data (“connect to Postgres” or “use S3”). Count the integrations that succeed first-try to approximate I_T . *Example:* Claude Code with MCP and Windsurf handle 10 + integrations; Junie and Amazon Q 4–5.
- 4) **Reasoning style:** Observe whether the tool decomposes tasks or responds monolithically. Iterative self-correction indicates Type B; orchestration with visible sub-agents signals Type C. *Example:* Claude Code behaves as Type C; Windsurf as Type B; Copilot as Type A.

B. Estimating autonomy score quickly

After these probes, plug observed determinants into Eq. (6) with approximate weights ($\alpha_1 = 0.3, \alpha_2 = 0.35, \alpha_3 = 0.25, \alpha_4 = 0.1$). An autonomy score above 7 implies high self-sufficiency; below 4 suggests the tool will need close supervision. *Practical shortcut:* If the tool independently executes one end-to-end workflow and maintains memory coherence, it likely exceeds $A_u > 7$.

C. Guidance for designers

To build tools with optimal autonomy:

- **Deepen context:** combine long-context attention with persistent RAG memory.
- **Controlled execution:** enable sandboxed shell or container operations with explicit rollback.
- **Integration by protocol:** adopt declarative interface specifications (e.g., MCP) for stable cross-system connectivity.
- **Distributed control:** design multi-agent reasoning loops for decomposable tasks.
- **Transparent feedback:** expose intermediate reasoning to foster trust and oversight.

These principles collectively push C_T, E_T, I_T, P_T toward optimal regions, maximising effective autonomy without sacrificing safety.

X. LIMITATIONS AND FUTURE WORK

Single-evaluator design limits generalisability; multi-rater replication and ablation testing are encouraged. The framework is language-agnostic but empirically grounded in Python contexts.

XI. CONCLUSION

A compact set of architectural determinants—contextual depth, execution scope, integration richness, and control topology—provides a practical, predictive theory of behaviour for AI coding assistants. The lightweight equations, heuristics, and

new rapid-assessment guide enable both researchers and engineers to evaluate and design assistants with greater autonomy and reliability.

ACKNOWLEDGEMENTS

Thanks to practitioners and collaborators who provided task corpora and feedback during the evaluation.

REFERENCES

- [1] A. Vaswani *et al.*, “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] T. B. Brown *et al.*, “Language models are few-shot learners,” *NeurIPS*, 2020.
- [3] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP,” *NeurIPS*, 2020.
- [4] M. Chen *et al.*, “Evaluating large language models trained on code,” *arXiv:2107.03374*, 2021.
- [5] S. Barke *et al.*, “Grounded copilot: How programmers interact with code-generating models,” *Proc. ACM Program. Lang.*, vol. 7, OOP-SLA1, 2023.
- [6] P. Vaithilingam *et al.*, “Expectation vs. experience: Evaluating the usability of code generation tools,” *CHI EA*, 2022.
- [7] J. Park *et al.*, “Generative agents: Interactive simulacra of human behaviour,” *arXiv:2304.03442*, 2023.
- [8] J. Leike *et al.*, “Scalable oversight for large language models,” *arXiv:2308.10704*, 2023.
- [9] E. Horvitz, “Principles of mixed-initiative user interfaces,” *CHI*, 1999.
- [10] M. Wooldridge and N. R. Jennings, “Intelligent agents: Theory and practice,” *Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.
- [11] Anthropic, “Model context protocol specification,” 2024. [Online]. Available: <https://modelcontextprotocol.io>
- [12] R. K. Yin, *Case Study Research and Applications*, 6th ed., Sage, 2017.